

Travelling Salesman Problem (TSP) Using Python

Aim

To implement the **Travelling Salesman Problem (TSP)** using Python and determine the minimum cost required for a salesman to visit all cities exactly once and return to the starting city.

Objective

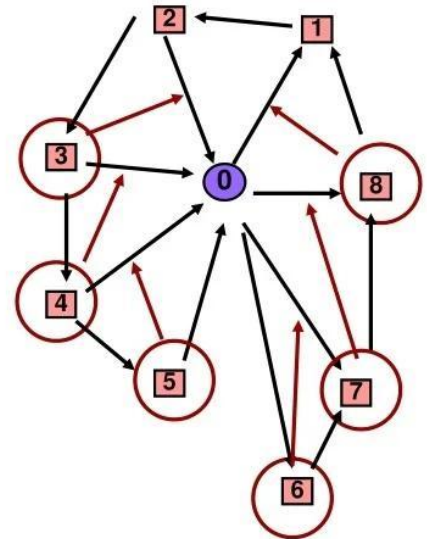
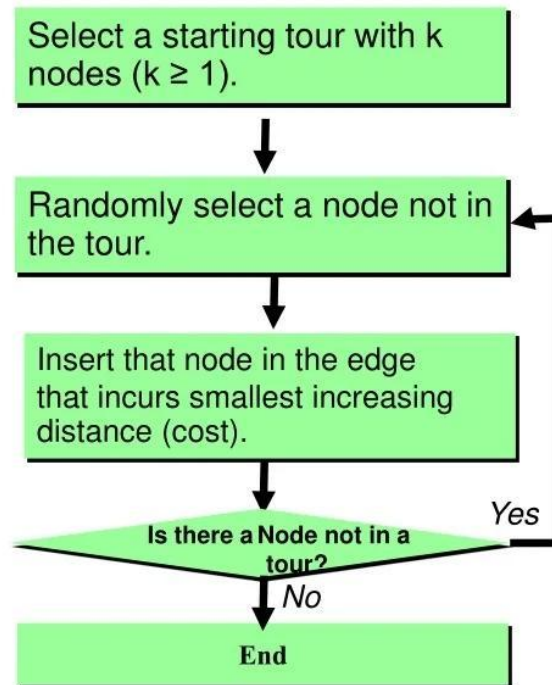
- To understand the Travelling Salesman Problem.
- To find the shortest possible route that visits each city exactly once.
- To calculate the minimum travel cost.
- To implement the solution using Python.
- To analyze the efficiency of the algorithm.

Algorithm

1. Start.
2. Define the cost matrix representing distances between cities.
3. Select the starting city.
4. Generate all possible permutations of the remaining cities.
5. For each possible route:
 - Calculate the total travel cost.
 - Compare it with the current minimum cost.
6. Store the route if its cost is smaller than the previous minimum.
7. Display the minimum cost and the best route.
8. Stop.

flowchart

Arbitrary Insertion Heuristic



Code:

```
from itertools import permutations
```

```
def travelling_salesman(graph, start):
```

```
    n = len(graph)
```

```
    cities = list(range(n))
```

```
    cities.remove(start)
```

```
    min_cost = float('inf')
```

```
    best_path = []
```

```
    for perm in permutations(cities):
```

```
        current_cost = 0
```

```
current_path = [start] + list(perm) + [start]
```

```
for i in range(len(current_path) - 1):
```

```
    current_cost += graph[current_path[i]][current_path[i + 1]]
```

```
if current_cost < min_cost:
```

```
    min_cost = current_cost
```

```
    best_path = current_path
```

```
return min_cost, best_path
```

```
graph = [
```

```
    [0, 10, 15, 20],
```

```
    [10, 0, 35, 25],
```

```
    [15, 35, 0, 30],
```

```
    [20, 25, 30, 0]
```

```
]
```

```
start_city = 0
```

```
cost, path = travelling_salesman(graph, start_city)
```

```
print("Minimum Cost:", cost)
```

```
print("Best Path:", " -> ".join(map(str, path)))
```

output:

```
Minimum Cost: 80
Best Path: 0 -> 1 -> 3 -> 2 -> 0
```

Result:

The Travelling Salesman Problem was successfully implemented using Python. The algorithm evaluated all possible routes and identified the route with the minimum travel cost. For the given cost matrix, the minimum cost obtained was **80**, and the optimal route was:

0 → 1 → 3 → 2 → 0

Conclusion:

The Travelling Salesman Problem was solved successfully using the brute-force approach in Python. The program generated all possible paths, calculated the total cost for each path, and selected the route with the minimum cost. Although this method guarantees the optimal solution, it becomes inefficient for a large number of cities due to its factorial time complexity. For larger datasets, more efficient algorithms such as Dynamic Programming, Branch and Bound, or Genetic Algorithms are recommended.